

조인

JOIN

조인(Join)이란 두 개 이상의 테이블을 서로 묶어서 하나의 결과를 만들어내는 것이다. 한 테이블 조회를 통해 결과를 얻는 경우도 있지만, 두 개 이상의 테이블을 엮어야만 원하는 형태가 나오는 경우도 많다.

1. 내부 조인 (INNER JOIN)

두 테이블을 연결할 때 가장 많이 사용되는 것이 **내부 조인**이다. 일반적으로 "조인"이라고 하면 **내부 조인**을 의미한다.

데이터베이스의 테이블은 하나로 구성되는 것보다는 여러 정보를 주제에 따라 분리해서 저장하는 것이 효율적이다. 그리고 이 분리된 테이블은 서로 **관계**를 맺고 있다.

두 테이블의 조인을 위해서는 일반적으로 PK-FK 관계로 맺어져야 하며, 이를 1:N 관계라고 부른다.

꼭 PK-FK 관계가 아니더라도 가능한 조인이 있다. 이를 상호 조인(CROSS JOIN)이라고 부른다. 상호 조인 외의 조인은 기본 키-외래 키 관계가 핵심이다.

1.1 내부 조인의 기본

일반적으로 조인이라고 부르는 것은 **내부 조인**을 말하는 것으로, 조인 중에서 가장 많이 사용된다. 조인은 3 개 이상의 테이블로도 가능하지만, 대부분 2개로 조인을 사용한다.

기본 구문

```
SELECT <열 목록>
FROM <첫 번째 테이블>
      INNER JOIN <두 번째 테이블>
      ON <조인될 조건>
[WHERE 검색 조건]
```

INNER JOIN 을 그냥 JOIN 이라고만 써도 INNER JOIN 으로 인식한다.

예제 데이터

members

id	name
1	홍길동
2	김철수

orders

id	member_id	product
1	1	노트북
2	1	키보드
3	2	모니터

쿼리

```
SELECT m.id, m.name, o.product
FROM members m
INNER JOIN orders o
    ON m.id = o.member_id;
```

결과

id	name	product
1	홍길동	노트북
1	홍길동	키보드
2	김철수	모니터

핵심 포인트

- INNER JOIN 은 두 테이블에서 매칭되는 값이 있는 행만 결과에 포함한다.
- `m.id = o.member_id` 조건에 맞는 데이터만 조회된다.
- 매칭되지 않는 데이터는 제외된다.

두 개의 테이블을 조인하는 경우 동일한 열 이름이 존재한다면 꼭 **테이블 이름.열이름** 형식으로 표기해야 한다.

1.2 내부 조인의 간결한 표현

`테이블_이름.열_이름` 형식으로 작성하면 어느 테이블에 속한 것인지 명확해지지만 코드가 너무 길어져서 오히려 복잡해 보인다. 이를 간결하게 표기하기 위해서는 FROM 절에 나오는 테이블 이름 뒤에 **별칭(alias)**을 줄 수 있다.

```
SELECT m.id, m.name, o.product
FROM members m
INNER JOIN orders o
    ON m.id = o.member_id;
```

- `members m`: members 테이블에 `m`이라는 별칭 부여

- orders o : orders 테이블에 o 라는 별칭 부여
- 이후 m.id , o.product 처럼 간결하게 사용 가능

1.3 내부 조인의 활용

예제 데이터

members

id	name	grade
1	홍길동	GOLD
2	김철수	SILVER
3	이영희	BRONZE

orders

id	member_id	order_date
1	1	2024-05-01
2	1	2024-05-10
3	2	2024-05-03

order_items

id	order_id	product	price
1	1	노트북	1500000
2	1	마우스	25000
3	2	키보드	80000
4	3	의자	120000

쿼리

```
SELECT
    m.id AS member_id,
    m.name,
    m.grade,
    SUM(oi.price) AS total_spent
FROM members m
INNER JOIN orders o
    ON m.id = o.member_id
INNER JOIN order_items oi
```

```
ON o.id = oi.order_id
WHERE m.grade = 'GOLD'
GROUP BY m.id, m.name, m.grade
HAVING SUM(oi.price) > 50000
ORDER BY total_spent DESC;
```

SQL 설명

요소	설명
INNER JOIN orders	회원이 실제로 주문한 주문 정보만 결합
INNER JOIN order_items	주문 안의 개별 상품 목록까지 결합
WHERE m.grade = 'GOLD'	GOLD 등급 회원만 필터링
SUM(oi.price)	모든 주문 상품 가격을 합산
GROUP BY	회원별로 집계
HAVING SUM(oi.price) > 50000	총 주문금액이 50,000원 초과하는 회원만
ORDER BY total_spent DESC	지출이 높은 순서대로 출력

결론: 내부 조인은 두 테이블에 모두 데이터가 있는 내용만 출력한다.

2. 외부 조인 (OUTER JOIN)

내부 조인은 두 테이블에 모두 결과가 있어야만 결과가 나오지만, **외부 조인**은 한쪽에만 데이터가 있어도 결과가 나온다.

2.1 외부 조인의 기본

외부 조인은 두 테이블을 조인할 때 필요한 내용이 한쪽 테이블에만 있어도 결과를 추출할 수 있다.

기본 구문

```
SELECT <열 목록>
FROM <첫 번째 테이블 (LEFT 테이블)>
    <LEFT | RIGHT | FULL> OUTER JOIN <두 번째 테이블 (RIGHT 테이블)>
ON <조인될 조건>
[WHERE 검색 조건];
```

LEFT OUTER JOIN 예제

```
SELECT m.id, m.name, o.product
FROM members m
```

```
LEFT JOIN orders o
  ON m.id = o.member_id;
```

왼쪽에 있는 테이블을 기준으로 외부 조인을 한다. LEFT OUTER JOIN 을 줄여서 LEFT JOIN 이라고 써도 된다.

LEFT OUTER JOIN은 왼쪽 테이블의 내용은 모두 출력되어야 한다. 기준 테이블을 보존하면서 다른 쪽 테이블 데이터가 있으면 붙이고 없으면 NULL을 넣어주는 방식이다.

RIGHT OUTER JOIN 예제

RIGHT OUTER JOIN 으로 동일한 결과를 출력하려면 단순히 왼쪽과 오른쪽 테이블의 위치만 변경해주면 된다.

```
SELECT m.id, m.name, o.product
FROM orders o
RIGHT JOIN members m
  ON m.id = o.member_id;
```

2.2 외부 조인의 활용

```
SELECT
  m.id AS member_id,
  m.name,
  m.grade,
  MAX(o.order_date) AS last_order_date
FROM members m
LEFT JOIN orders o
  ON m.id = o.member_id
GROUP BY m.id, m.name, m.grade
ORDER BY last_order_date DESC;
```

SQL 설명

요소	설명
LEFT JOIN orders	주문이 없는 회원도 유지
MAX(o.order_date)	가장 최근 주문일만 가져오기
GROUP BY	회원별로 하나의 행으로 묶음
last_order_date DESC	최근 주문 기준 정렬
주문 없는 회원은 last_order_date 가 NULL	LEFT JOIN의 핵심

2.3 FULL OUTER JOIN

FULL OUTER JOIN은 왼쪽 외부 조인과 오른쪽 외부 조인이 합쳐진 것이다. 어느 쪽이든 한쪽에 들어있는 내용이면 출력한다.

```
SELECT m.id, m.name, o.product
FROM members m
FULL OUTER JOIN orders o
    ON m.id = o.member_id;
```

- 왼쪽 테이블에만 있는 데이터도 출력
- 오른쪽 테이블에만 있는 데이터도 출력
- 양쪽 모두에 있는 데이터도 출력
- 매칭되지 않는 쪽은 NULL로 표시

MySQL은 FULL OUTER JOIN을 직접 지원하지 않으므로, LEFT JOIN과 RIGHT JOIN을 UNION으로 결합하여 구현해야 한다.

3. 기타 조인

3.1 상호 조인 (CROSS JOIN)

상호 조인은 한쪽 테이블의 모든 행과 다른쪽 테이블의 모든 행을 조인시키는 기능을 말한다.

왼쪽 테이블의 모든 행 × 오른쪽 테이블의 모든 행 = 모든 조합 생성

기본 구문

```
SELECT *
FROM members
CROSS JOIN orders;
```

특징

- 상호 조인은 ON 구문을 사용할 수 없다.
- 결과의 내용은 비즈니스 로직 측면에서 의미가 없다. 랜덤으로 조인하기 때문이다.
- 상호 조인의 주 용도는 테스트하기 위해 대용량의 데이터를 생성할 때이다.

예제: 테스트 데이터 생성

```
-- members 3개 행, orders 100개 행이면
-- CROSS JOIN 결과는 3 × 100 = 300개 행
CREATE TABLE test_data AS
SELECT m.id, m.name, o.product
FROM members m
CROSS JOIN orders o;
```

3.2 CREATE TABLE ~ SELECT

새로운 테이블을 만들면서 동시에 SELECT 결과를 그 테이블에 넣는 SQL 문법이다.

```
CREATE TABLE gold_members AS
SELECT id, name, grade
FROM members
WHERE grade = 'GOLD';
```

구문 설명

부분	의미
CREATE TABLE gold_members AS	새로운 테이블 gold_members 생성
SELECT id, name, grade	기존 테이블에서 선택할 컬럼 지정
FROM members	복사할 대상 테이블
WHERE grade = 'GOLD'	복사할 데이터 조건 지정

주의할 점

- 원본 테이블의 PK / FK / 인덱스 / AUTO_INCREMENT 속성은 복사되지 않는다.
- 컬럼과 데이터만 복사된다.
- 필요하다면 이후에 별도로 PRIMARY KEY 등을 설정해줘야 한다.

3.3 자체 조인 (SELF JOIN)

자체 조인은 자신이 자신과 조인한다는 의미이다. 주로 계층 구조(조직도, 대댓글, 카테고리 트리 등)를 표현할 때 사용된다.

예제 데이터

employees

id	name	manager_id
1	홍길동	NULL
2	김철수	1
3	이영희	1
4	박민수	2

쿼리

```
SELECT
    e.name AS employee_name,
    m.name AS manager_name
FROM employees e
LEFT JOIN employees m
    ON e.manager_id = m.id;
```

결과

employee_name	manager_name
홍길동	NULL
김철수	홍길동
이영희	홍길동
박민수	김철수

핵심 포인트

- 하나의 테이블에 상하 관계가 있을 때 자체 조인을 많이 사용한다.
- 자기 자신을 두 번 불러오므로 별칭이 필수다.
- e 는 직원, m 은 매니저로 구분하여 동일 테이블을 다른 역할로 사용한다.

조인 유형 정리

조인 유형	설명	특징
INNER JOIN	양쪽 테이블에 매칭되는 데이터만 출력	가장 많이 사용
LEFT OUTER JOIN	왼쪽 테이블 기준, 오른쪽은 매칭 안되면 NULL	기준 테이블 모든 데이터 보존
RIGHT OUTER JOIN	오른쪽 테이블 기준, 왼쪽은 매칭 안되면 NULL	LEFT JOIN의 반대
FULL OUTER JOIN	양쪽 테이블의 모든 데이터 출력	MySQL 미지원 (UNION 활용)
CROSS JOIN	모든 조합 생성 (카티션 곱)	테스트 데이터 생성용
SELF JOIN	자기 자신과 조인	계층 구조 표현